

```

import com.fasterxml.jackson.databind.DeserializationFeature;
import com.fasterxml.jackson.databind.MappingIterator;
import com.fasterxml.jackson.databind.ObjectMapper;
import com.fasterxml.jackson.dataformat.yaml.YAMLFactory;

import javax.imageio.ImageIO;
import javax.swing.*;
import java.awt.*;
import java.io.File;
import java.io.FileInputStream;
import java.io.InputStream;
import java.net.URL;
import java.util.*;

public class Main {
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new MainMenu().open());
    }
}

class MainMenu extends JFrame {
    public MainMenu() {
        setTitle("Multiple-Choice Quizzes");
        setSize(1280, 720);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        JButton startButton = new JButton("Start Game");
        startButton.setFont(new Font("Arial", Font.PLAIN, 20));
        startButton.setPreferredSize(new Dimension(300, 60));

        JPanel panel = new JPanel();
        setLayout(new GridBagLayout());
        panel.add(startButton, new GridBagConstraints());

        getContentPane().add(panel);
    }
}

```

```

startButton.addActionListener(e -> {

    QuestionManager questionManager = QuestionManager.getInstance();

    Question question = questionManager.getNextQuestion();

    new QuestionMenu(question).open();

    dispose();

});

}

public void open() {

    setVisible(true);

}

}

class QuestionMenu extends JFrame {

    public QuestionMenu(Question question) {

        setTitle("Multiple-Choice Quizzes");

        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        setSize(1280, 720);

        JPanel panel = new JPanel();

        JScrollPane scrollPane = new JScrollPane();

        panel.setLayout(new GridBagLayout());

        scrollPane.setViewportView(panel);

        JLabel questionLabel = new JLabel(question.getQuestion());

        questionLabel.setFont(new Font("Arial", Font.PLAIN, 20));

        panel.add(questionLabel, constraints());

        JLabel pointsLabel = new JLabel("Total Points: " +
QuestionManager.getInstance().getTotalPoints());

        pointsLabel.setFont(new Font("Arial", Font.PLAIN, 20));

        panel.add(pointsLabel, constraints());

```

```

        if (!question.getImageUrl().isEmpty() && question.getImageUrl() != null &&
!question.getImageUrl().isBlank()) {
            try {
                ImageIcon imageIcon = new ImageIcon(ImageIO.read(new
URL(question.getImageUrl())));
                Image image = imageIcon.getImage().getScaledInstance(300, 300,
Image.SCALE_SMOOTH);
                ImageIcon scaledImageIcon = new ImageIcon(image);
                JLabel imageLabel = new JLabel(scaledImageIcon);
                imageLabel.setPreferredSize(new Dimension(300, 300));
                panel.add(imageLabel, constraints());
            } catch (Exception e) {
                throw new IllegalArgumentException("Invalid image URL for question: " +
question.getQuestion() + " - " + question.getImageUrl());
            }
        }

        for (String answer : question.getAllAnswers()) {
            JButton answerButton = new JButton(answer);
            answerButton.setFont(new Font("Arial", Font.PLAIN, 20));
            answerButton.setPreferredSize(new Dimension(300, 60));
            panel.add(answerButton, constraints());

            answerButton.addActionListener(e -> {
                if (question.getCorrectAnswers().contains(answer)) {
                    JOptionPane.showMessageDialog(this, "Your answer is correct! You have received
" + question.getPoints() + " points! " + question.getElaboration(), "Correct",
JOptionPane.INFORMATION_MESSAGE);
                    QuestionManager.getInstance().addPoints(question.getPoints());
                } else {
                    JOptionPane.showMessageDialog(this, "Your answer is incorrect! " +
question.getElaboration(), "Incorrect", JOptionPane.ERROR_MESSAGE);
                }
                QuestionManager questionManager = QuestionManager.getInstance();
                Question nextQuestion = questionManager.getNextQuestion();
                if (nextQuestion != null) {
                    new QuestionMenu(nextQuestion).open();
                } else {
                    JOptionPane.showMessageDialog(this, "No more questions available!", "Game
Over", JOptionPane.INFORMATION_MESSAGE);
                }
            });
        }
    }
}

```

```

        QuestionManager.getInstance().reset();

        new MainMenu().open();

    }

    dispose();

});

}

JButton skipButton = new JButton("Skip");
skipButton.setFont(new Font("Arial", Font.PLAIN, 20));
skipButton.setPreferredSize(new Dimension(300, 60));
panel.add(skipButton, constraints());
skipButton.addActionListener(e -> {

    QuestionManager questionManager = QuestionManager.getInstance();
    Question nextQuestion = questionManager.getNextQuestion();
    if (nextQuestion != null) {
        new QuestionMenu(nextQuestion).open();
    } else {
        JOptionPane.showMessageDialog(this, "No more questions available!", "Game Over",
JOptionPane.INFORMATION_MESSAGE);

        QuestionManager.getInstance().reset();
        new MainMenu().open();

    }

    dispose();

});

getContentPane().add(panel);

}

private GridBagConstraints constraints() {
    GridBagConstraints constraints = new GridBagConstraints();
    constraints.gridx = 0;
    constraints.fill = GridBagConstraints.HORIZONTAL;
    constraints.gridy = GridBagConstraints.RELATIVE;
    constraints.insets = new Insets(7, 7, 7, 7);
    return constraints;
}

```

```

    public void open() {
        setVisible(true);
    }
}

class QuestionManager {
    private static QuestionManager INSTANCE;
    private final List<Question> questions;
    private Iterator<Question> questionIterator;
    private int totalPoints;

    private QuestionManager() {
        ObjectMapper mapper = new ObjectMapper(new YAMLFactory());
        mapper.configure(DeserializationFeature.FAIL_ON_UNKNOWN_PROPERTIES, false);
        try (InputStream stream = new FileInputStream("questions.yml")) {
            MappingIterator<Question> questionsIterator =
mapper.readerFor(Question.class).readValues(stream);
            questions = questionsIterator.readAll();
            INSTANCE = this;
        } catch (Exception e) {
            throw new IllegalArgumentException("Invalid questions file: " + e.getMessage());
        }
        try {
            File file = new File("savedata.txt");
            if (file.exists()) {
                Scanner scanner = new Scanner(file);
                totalPoints = scanner.nextInt();
                scanner.close();
            } else {
                totalPoints = 0;
            }
        } catch (Exception e) {
            throw new IllegalArgumentException("Invalid points for questions: " + e.getMessage());
        }
    }

    public static QuestionManager getInstance() {

```

```

        return Objects.requireNonNullElseGet(INSTANCE, QuestionManager::new);
    }

    public Question getNextQuestion() {
        if (questionIterator == null) reset();
        if (questionIterator.hasNext()) return questionIterator.next();
        else return null;
    }

    public void reset() {
        questionIterator = questions.iterator();
    }

    public void addPoints(int points) {
        totalPoints += points;
        try {
            File file = new File("savedata.txt");
            if (!file.exists()) {
                file.createNewFile();
            }
            Formatter formatter = new Formatter(file);
            formatter.format("%d", totalPoints);
            formatter.close();
        } catch (Exception e) {
            throw new IllegalArgumentException("Invalid points for questions: " + e.getMessage());
        }
    }

    public int getTotalPoints() {
        return totalPoints;
    }
}

class Question {
    private String question;
    private List<String> incorrectAnswers;

```

```
private List<String> correctAnswers;

private String imageUrl;

private String elaboration;

private int points;

public String getQuestion() {
    return question;
}

public List<String> getIncorrectAnswers() {
    return incorrectAnswers;
}

public List<String> getCorrectAnswers() {
    return correctAnswers;
}

public String getImageUrl() {
    return imageUrl;
}

public String getElaboration() {
    return elaboration;
}

public int getPoints() {
    return points;
}

public List<String> getAllAnswers() {
    List<String> allAnswers = new ArrayList<>(incorrectAnswers);
    allAnswers.addAll(correctAnswers);
    Collections.shuffle(allAnswers);
    return allAnswers;
}
}
```